Reasoning Mode Breaks the Plateau: Verified LLM-Evolutionary Search Matches Three Decades of Best-Known Covering Designs

KEREM TÜRKYILMAZ, Independent Researcher, Türkiye

LLM-driven evolutionary program search has produced striking mathematical constructions, but the literature says little about *which model capability does the breakthrough work*. We study this question on covering designs $C(v, k, t)$ — a classical construction domain untouched by the FunSearch/AlphaEvolve lineage — using a problem-agnostic evolution harness whose defining property is that every score is recomputed by a mathematically exact verifier and no claim rests on model or solver self-report. Starting from a generic greedy solver, our campaign improved $C(32, 8, 4)$ from 1,258 to 977 blocks and then stalled: across 75 iterations and three configurations, including full rewrites by the flagship model with reasoning disabled, no mutation left the plateau. With extended reasoning enabled — and nothing else changed — the same model broke the plateau in its first improving iteration by writing a *general* affine-geometry construction, matching the best-known value of 620 blocks, unimproved since 1996. A controlled repetition from the frozen plateau state separates the arms categorically: reasoning-on 3/3 breakthroughs, each independently reaching exactly 620; reasoning-off 0/3 (one-sided $p = 0.05$; effect −36%). Under a pre-registered 29-cell benchmark the single evolved program matched 22 best-known values — including four in cells where its discovered construction is inapplicable — with every tie re-verified by an independent verifier and high cross-seed stability (25/29 identical triples). Direct-recall probes show the model can neither state the target value nor emit the artifact without writing code, dissociating memorization from the reasoning-to-code pathway. We release all certificates, frozen protocols, raw logs, and negative results.

CCS Concepts: • **Computing methodologies** → **Genetic programming**; *Natural language generation*; • **Mathematics of computing** → *Combinatorics*.

Additional Key Words and Phrases: LLM-driven evolution, program search, covering designs, extended reasoning, exact verification, FunSearch, AlphaEvolve

ACM Reference Format:
Kerem Türkyılmaz. 2026. Reasoning Mode Breaks the Plateau: Verified LLM-Evolutionary Search Matches Three Decades of Best-Known Covering Designs. *ACM Trans. Evol. Learn. Optim.* 0, 0, Article 0 (August 2026), 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

In late 2023, FunSearch demonstrated that a large language model inside an evolutionary loop can produce genuinely new mathematics [Romera-Paredes et al. 2024]; AlphaEvolve scaled the recipe to dozens of problems and whole codebases [Novikov et al. 2025]. These systems are typically reported as monoliths: a model, a mutation scheme, an evaluator, a database — and a result. When the result is good, it is hard to say which ingredient earned it. Practitioners inherit an expensive

Author's Contact Information: Kerem Türkyılmaz, Independent Researcher, Türkiye, 3kerem@gmail.com.

ACM 2688-3007/2026/8-ART0
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ambiguity: should the next budget go to more iterations, a bigger population, a stronger model — or to a different *mode* of the same model?

This paper isolates one ingredient experimentally: the model's extended reasoning mode ("thinking"), in which the model produces a long private chain of thought before its answer. Our vehicle is a campaign on covering designs — minimum collections of k-element blocks covering all t-subsets of a v-element universe — chosen for three properties. First, *verifier asymmetry*: constructing good coverings has occupied specialists for decades, but verifying a candidate is exact integer counting, which suits our harness's governing rule that nothing an evolved program says about itself is ever trusted. Second, the domain has a canonical, curated frontier (the La Jolla Covering Repository and its live successor [Gordon 2026]) whose values on our target cells have stood for 15–30 years — a stagnant frontier against which even exact ties are informative. Third, to our knowledge no system in the FunSearch lineage has touched it.

The campaign itself furnished the experiment. Cheap ensemble evolution improved $C(32, 8, 4)$ from a 1,258-block seed baseline to 977 blocks within five mutations, then plateaued. We spent 75 iterations establishing that the plateau was real rather than an instrument artifact: we repaired a mutation-waste pathology (48% of calls were being rejected before evaluation), verified the fix (waste fell to 8%), and escalated to the conventional remedy — full rewrites by the flagship model — which produced children clustering to the *exact* plateau value from four independent rewrites. Then we enabled extended reasoning on the same model, same prompt, same frozen population. The first improving iteration rewrote the solver around a general finite-geometry construction — all d-flats of $\mathrm{AG}(m, p)$ for any prime-power universe — whose block count at $(32, 8, 4)$ is exactly 620: the repository's best-known value, dated 1996, and 357 blocks (36%) below the plateau. A controlled repetition experiment from the frozen state confirmed the effect is categorical, not luck: three of three reasoning-enabled slices rediscovered the construction independently; zero of three reasoning-disabled slices left the plateau band ($p = 0.05$, one-sided Fisher; 55 disabled full-rewrite attempts in total never produced a structural change).

We then asked what the discovery is worth beyond its cell. Under a benchmark protocol frozen before execution — 29 cells spanning proven-optimum gates, affine-family holdouts, and cells where the affine construction cannot apply — the single evolved program matched the best-known value in 22 cells with zero per-cell adaptation, including $C(49, 8, 2)$, where the pure construction is seven blocks short and the evolved local search closes the gap, and four non-prime-power cells served purely by the evolved search machinery. All ties were re-verified by a standalone verifier sharing no code with the harness. Because "the model memorized the tables" is the natural objection, we also ran dissociation probes: the deployed model, asked directly, declines to state the value of $C(32, 8, 4)$; and asked to emit a covering as plain text with a 64k-token reasoning budget but no code, it produces zero blocks. The capability that matters is not recall but the reasoning-to-code-to-execution pathway.

Our contributions:

- **C1 — A controlled reasoning-mode experiment in an evolutionary discovery loop** (Section 6): to our knowledge the first, showing a categorical 3/3-vs-0/3 breakthrough separation with a 36% objective effect, against a backdrop of mixed reasoning-ablation results in other task families.
- **C2 — Verified rediscovery at a stagnant frontier** (Sections 5, 7): 22 of 29 pre-registered cells tied to best-known values by one evolved program, every tie independently verified; covering designs enter the LLM-evolutionary literature.
- **C3 — An exactness-first, problem-agnostic harness** (Sections 3–4): plugin contract with fitness/cost separation, multi-seed fitness, work-counter determinism, solution archiving,

and leak-audited prompts — each mechanism motivated by a documented failure; plus engineering findings (mutation-waste dynamics, an ensemble-seeding pathology) reported for reuse.

C4 — A contamination-dissociation methodology (Section 8): value-recall, direct-artifact, and code-pathway probes that calibrate what "rediscovery" means when the literature is in the weights.

Section 2 reviews related work; Section 9 details limitations — foremost that all positive results are ties, not records, and that the causal claim is scoped to one model, one plateau, one domain. Everything needed to re-verify or extend this work — harness, certificates, protocols, raw logs — is released as open source.

2 Related Work

2.1 LLM-driven evolutionary program search

FunSearch [Romera-Paredes et al. 2024] established that pairing an LLM mutation operator with a programmatic evaluator can produce new mathematical constructions (cap sets, bin-packing heuristics). AlphaEvolve [Novikov et al. 2025] generalized the recipe to whole codebases and reported results across more than fifty mathematical problems — rediscovering the best known construction in ∼75% of them and improving it in ∼20%, a framing that makes *rediscovery rate* an accepted currency of this literature and one we adopt. Follow-on systems refine the search architecture: ShinkaEvolve [Lange et al. 2025] targets sample efficiency (circle packing, Heilbronn triangles, autocorrelation inequalities), CodeEvolve [Assumpção et al. 2025] provides an open implementation, Nagda et al. apply reinforced generation to Ramsey numbers [Nagda et al. 2026], explicitly reporting recovery of known bounds alongside improvements, and Bhan et al. extend the approach to Zarankiewicz numbers [Bhan et al. 2026]. Negative results have also begun to appear (bijection discovery with OpenEvolve remaining hard [Brown et al. 2025]), which we take as a healthy norm and follow in Sections 7 and 9. Analyses of *why* these loops work concentrate on the evolutionary component [Zhang et al. 2024]; the contribution of specific *model capabilities* — in particular extended reasoning — has, to our knowledge, not been isolated experimentally in a discovery loop. Across all published problem lists of this lineage we find no covering-design instances; both gaps are addressed here.

2.2 Reasoning-mode ablations

Controlled comparisons of LLMs with reasoning enabled versus disabled exist outside discovery loops, with strikingly mixed outcomes: reasoning helps handwriting-synthesis agents [Sesay et al. 2026] and long-form information-control tasks [He et al. 2025], is neutral for prompt-attack detection [Le et al. 2026], and *degrades* content-moderation accuracy [Dong et al. 2026]. This task-dependence is the backdrop against which our result should be read: in the structural-paradigm-shift regime of Section 6, the effect is not a few points of accuracy but a categorical 3/3-versus-0/3 separation with a 36% objective improvement.

2.3 Covering designs

Upper bounds for covering numbers $C(v, k, t)$ are curated by the La Jolla Covering Repository and its live successor [Gordon 2026]. The founding constructions paper [Gordon et al. 1995] combined greedy methods, finite-geometry constructions (including AG(m, p) flats — the family our loop rediscovers), and synthesis rules; subsequent stochastic-search work (simulated annealing [Nurmela and Östergård 1993], cooperative tabu search [Dai et al. 2006]) improved many small cells. The specific cells we target have been stable for 15–30 years, and we find no post-2020 method wave

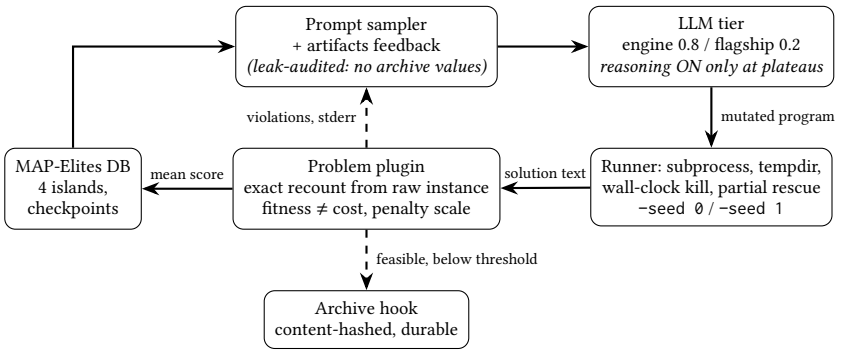

Fig. 1. The harness. The exactness boundary (bottom) recomputes all scores from raw instances; solver self-reports never cross it.

touching them — a stagnant-frontier setting that makes exact ties informative even though they set no records.

3 The Discovery Harness

Our harness descends from the FunSearch/AlphaEvolve template — an LLM proposes program mutations, an evaluator scores them, an evolutionary database (MAP-Elites over islands; we use OpenEvolve [ope 2025]) maintains the population — but its design center is different: *every scored value is recomputed by a mathematically exact instrument, and nothing the evolved program says about itself is trusted* (Figure 1). This section describes the contract that enforces this, and the design decisions that were each paid for by a concrete failure earlier in the project.

3.1 Problem plugins and the exactness contract

The core never mentions any concrete problem. A problem is a plugin exposing four members: a SENSE flag (min/max), a strict instance parser, an evaluator mapping solution text to a verdict, and a penalty scale derived from the instance. Two asymmetric error policies apply: a malformed *instance* raises immediately (instrument fault — the loop must halt), while a malformed *solution* never raises — it produces a graded, penalized verdict, because candidate programs will emit garbage and the loop must keep moving. The verdict separates *fitness* (penalized, for evolution) from *cost* (the claim-bearing objective, defined only for feasible solutions); conflating the two is a classic reward-hacking door and is structurally closed here. The sign convention between SENSE and the maximizing evolution engine is applied in exactly one function — an inverted sign silently reverses evolution, so this conversion is the most heavily unit-tested line of the codebase.

Evolved programs run in a separate process with a temporary working directory and a wall-clock kill; whatever partial output exists at timeout is rescued and graded (all our solvers are anytime by contract). The evaluator recounts everything from the raw instance with exact arithmetic — exact rational arithmetic for the reliability case study, exact integer counting for covering. A solver's self-reported objective is ignored, but *whether it matches* the recount is logged as a silent honesty sensor. Penalty scales are derived from instance data such that no constraint violation can ever be profitable; the test suite asserts the invariant "every infeasible verdict scores strictly below every feasible one" per plugin.

3.2 Multi-seed fitness and the determinism contract

Two additions were forced by a painful incident. In an earlier record-hunting run on $C(28, 9, 3)$, a 56-block solution was observed once by the evaluator, was not persisted, and could not be reproduced afterwards: the genome derived internal phase budgets from wall-clock fractions, so even fixed-seed reruns diverged under machine-load variation (measured same-seed spread on that genome: 63–73 blocks). The episode produced three mechanisms. First, *solution archiving*: an environment-gated hook persists every feasible solution passing a cost threshold to durable storage under a content-hashed filename (idempotent under parallel evaluation, never able to raise into the evaluation path). Second, *multi-seed fitness*: the adapter can run a candidate once per seed in a declared list, passing `-seed N` on the CLI; the evolutionary fitness is the *mean* of combined scores over seeds, the reported cost is the best run's. A program that is only occasionally lucky is thereby scored as such — fitness measures the program, not one machine-moment. Third, a *determinism contract* in the mutation prompt: internal phase budgets must be derived from work counters (iterations, node counts), never from wall-clock fractions, with wall-clock permitted only as the final hard stop. Section 7's seed-stability result (identical costs across three seeds in 25 of 29 cells) is the empirical yield of this contract.

3.3 Problem-agnosticism as a tested property

The harness's generality is not aspirational but regression-tested: after the initial reliability-design case study (weighted k-out-of-n systems, where small instances have *provably* optimal solutions via exhaustive enumeration, giving the evaluator itself a measurable accuracy), a cap-set plugin (the FunSearch precedent problem) and the covering-design plugin of this paper were each added with zero changes to the core — the project treats any needed core edit as a leak to be fixed, not accommodated. We disclose one accepted limitation: on the Windows host used for these experiments, the candidate subprocess is not OS-sandboxed (no network/memory isolation enforcement); the risk is accepted for a single-user machine and the setup is portable to containerized execution.

4 Case Study Instrument: Covering Designs

4.1 Problem and verifier asymmetry

A covering design $C(v, k, t)$ is a collection of k-element blocks from a v-element universe such that every t-element subset of the universe is contained in at least one block; the objective is to minimize the number of blocks. The domain exhibits the *verifier asymmetry* that guides all our problem choices: constructing good coverings has occupied specialists for decades, but verifying a candidate is exact and cheap — count, over all $\binom{v}{t}$ t-subsets, whether each is covered. Our evaluator does exactly this count with integer arithmetic; there is no floating point anywhere in the scoring path.

4.2 Scoring without leakage

Feasible solutions with B blocks score $\text{Schonheim}(v, k, t)/B \in (0, 1]$, where the Schönheim bound [Schönheim 1964] — a classical, instance-computable lower bound — serves as the normalizer. This choice matters for hygiene: the normalizer injects *no empirical knowledge* (no archive values, no best-known sizes reach the prompt, the config, or the scoring function; repository values live in a reference directory that the loop cannot read). A score of 1.0 means the Schönheim bound is met, which is rarely possible on large cells; scores are comparable across cells, enabling multi-instance fitness sets. Infeasible outputs map into a disjoint band below every feasible score, graded by the fraction of uncovered t-subsets — an early timeout that leaves a partial covering still receives gradient. Format violations (wrong cardinality, out-of-range or duplicate elements,

duplicate blocks) poison the solution but never crash the evaluator, and work caps guarantee the instrument terminates on adversarial inputs.

4.3 Calibrating the instrument against theory and ground truth

Before any evolution we calibrated the instrument in three layers. *Curation*: the repository's public data dump was reduced to a targets table (8,759 cells), validated by hard invariants — the Schönheim bound never exceeds the recorded lower bound, lower bounds never exceed recorded sizes, and on the 96 cells with $k = 3, t = 2$ the recorded sizes equal the exact Fort–Hedlund value [Fort and Hedlund 1958] (96/96). The same pass surfaced 196 rows of history-hygiene anomalies in auxiliary fields, fixing the project rule that archive comparisons always target the size field. *Ground truth*: an exhaustive enumerator (iterative deepening on the block count with first-uncovered-subset branching and a $\lceil \text{uncovered}/\binom{k}{t} \rceil$ bound) independently re-proved nine archive-proven cells, including refuting the Schönheim value 11 for $C(7, 4, 3)$ and certifying 12 (450k search nodes), and proving the gate cell $C(13, 3, 2) = 26$ in milliseconds. These proven cells act as *gates* in every later experiment: an evolved program failing to reach a proven optimum signals instrument or population damage before any claim is risked. *Baseline*: the human-written seed solver (sampled-candidate greedy, redundancy elimination, ruin-and-recreate; anytime with atomic output replacement) reaches the proven optima on the gates but sits far behind the archive on target cells — e.g., 1,269 vs. 620 blocks on $C(32, 8, 4)$ at small budgets — establishing, via the archive values as reachability certificates, that genuine headroom exists for evolution rather than a saturated objective.

5 The Evolution Campaign

This section narrates the $C(32, 8, 4)$ marathon chronologically, because the sequence itself — fast early gains, a genuine plateau, a failed conventional escalation, and a reasoning-mode breakthrough — is the experimental substrate for Section 6 (Figure 2). An earlier week of the same campaign, using pre-breakthrough artifacts and focused single-cell slices, had already matched five other decades-old repository values (including $C(28, 9, 3) = 56$, three independent certificates); we reference those results as context but the present narrative concerns the marathon cell $C(32, 8, 4)$, whose archive value (620, dated 1996) stood 396 blocks below our best artifact at the campaign's start.

5.1 Setup and early gains

The marathon configuration follows an AlphaEvolve-style two-model ensemble — a cost-efficient engine model (GLM-5, weight 0.8) and a flagship (Claude Opus 5, weight 0.2), both mutating via search-replace diffs — over a single-cell fitness set with two-seed mean fitness and a 300 s solver budget (Section 3). Two pre-launch measurements shaped the design. First, the strongest prior artifact silently capped its internal time budget at 50 s; with the cap lifted, six times more wall-clock *alone* did not improve quality (1,016 vs. 1,013 blocks) — wall-clock-fraction phase budgets do not scale, so the additional budget had to be made exploitable by evolution. Second, a five-iteration smoke run immediately improved the two-seed score from 0.5181 to 0.5423 (978 blocks, later re-measured at 977), beating the previous all-time artifact within five mutations and validating the ensemble plumbing end-to-end.

5.2 The plateau, instrumented

The next 50 ensemble iterations produced *zero* improvements — but attributing this to a search plateau required first eliminating an instrumentation artifact. In the first 25-iteration slice, 48% of LLM calls were wasted: 36% produced children exceeding the harness's maximum program length (the genome had grown to ~30k characters, so additive diffs pierced the 35k cap), and 12% were

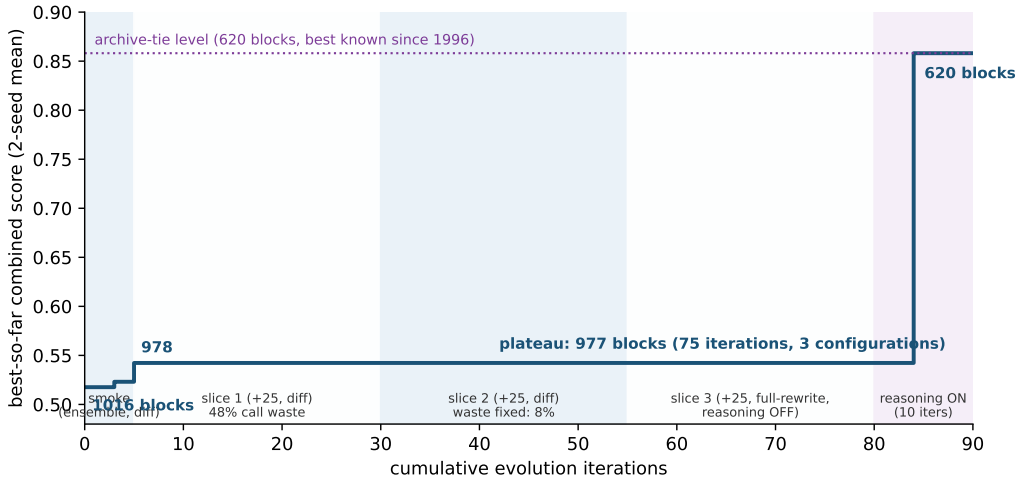

Fig. 2. Best-so-far combined score (two-seed mean) over the marathon. The plateau at 977 blocks survives 75 iterations across three configurations; the reasoning-enabled slice breaks it to the archive-tie level (620 blocks) in its first improving iteration.

unparseable diffs. Raising the cap to 45k and adding an explicit pruning instruction to the mutation contract ("prefer replacing code over accumulating it") cut waste to 8% in the second slice — which still yielded zero improvements. The conventional escalation, a 25-iteration full-rewrite slice with the reasoning-disabled flagship (the exact recipe that had broken an earlier plateau on $C(28, 9, 3)$), also failed: 24/25 valid rewrites, of which four independently returned to *exactly* 0.5423/977 and the rest clustered in 977–980. At this point the plateau was real and instrument-clean: 75 iterations, three configurations, one attractor.

A software finding from this phase is worth reporting for users of similar frameworks: the evolution framework seeds each worker process's model-selection RNG identically, so every worker draws the same model sequence — in short runs the configured 0.8/0.2 ensemble mix collapses (our five-iteration smoke run sent 5/5 calls to the 20%-weight model; we verified the effect end-to-end via proxy request logs and reproduced the draw sequence analytically). Over long runs the mix converges to the configured weights (21.3% over each worker's 75-draw prefix), but short diagnostic slices systematically misrepresent ensemble behavior.

5.3 The breakthrough and what was discovered

The reasoning-enabled slice that followed (motivation and controls in Section 6) broke the plateau in its first improving iteration: combined score 0.5423 → 0.8581, cost 977 → 620, matching the archive value to the block. The discovered program deserves description (Figure 3). The new routine detects whether $v = p^m$ for a small prime p; if so it constructs the translation group of the affine geometry $\mathrm{AG}(m, p)$ by digit-wise addition modulo p, enumerates all d-dimensional linear subspaces by repeated closure — d chosen maximal with $p^d \le k$ — and emits every coset (d-flat) as a block, padding blocks with random extra points when $k > p^d$. For $(v, k, t) = (32, 8, 4)$: $p = 2$, $m = 5$, $d = 3$, giving all 3-flats of $\mathrm{AG}(5, 2)$. Correctness is structural — any 4 points span an affine subspace of dimension ≤ 3 and hence lie in some 3-flat — and the count is exact: $p^{m-d} \begin{bmatrix} m \\ d \end{bmatrix}_p = 4 \cdot 155 = 620$ blocks. This is a classical construction; the 1996 archive value almost certainly descends from

```python
def affine_blocks(v, k, t, rng):
    """All d-flats of AG(m,p) as k-blocks, or None if not applicable."""
    if v > 512:
        return None
    p = None
    for q in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31):
        if v % q == 0:
            p = q
            break
    if p is None:
        return None
    m, x = 0, 1
    while x < v:
        x *= p
        m += 1
    if x != v:
        return None
    d, y = 0, 1
    while y * p <= k:
        y *= p
        d += 1
    if d < t - 1:
        return None

    # digit-wise addition mod p == the translation group of AG(m,p)
    add = []
    for a in range(v):
        row = [0] * v
        for b in range(v):
            s, mul, aa, bb = 0, 1, a, b
            for _ in range(m):
                s += (((aa % p) + (bb % p)) % p) * mul
```

Fig. 3. The evolved `affine_blocks` routine (first half; full listing in the repository). It derives the affine-geometry construction for arbitrary prime-power universes; no block lists or table values are stored.

the same family (the founding repository paper [Gordon et al. 1995] explicitly searched AG(m, p) flats). Three properties matter for our claims: the code contains *no* stored blocks or table values (its only dense numeric literal is a list of small primes); it is *general*, deriving the construction for any prime-power universe, which Section 7 exploits; and it was produced under a loop that never saw the number 620 in any input. The solution was captured by the archiving hook in triplicate, re-verified by the standalone verifier (35,960 of 35,960 t-subsets covered), and cross-checked against the live successor of the frozen repository on the same day.

5.4 Cost accounting

For reproducibility budgeting: the marathon consumed approximately 115 engine-model calls (diff mode, ∼3.4k output tokens each), 85 reasoning-disabled flagship calls (full rewrites, 7–9k tokens), and 40 reasoning-enabled flagship calls (∼48k output tokens each, dominated by reasoning traces), plus roughly 60 CPU-hours of evaluation. The entire discovery phase — from campaign start to the verified 620 — fit in under three days of wall-clock time on one desktop machine.

6 Controlled Experiment: Reasoning Mode as the Plateau Breaker

6.1 Motivation and design

The campaign of Section 5 left the population in a stable plateau: after 75 iterations (50 ensemble iterations followed by 25 full-rewrite iterations with the reasoning-disabled flagship model), the best program scored 0.5423 (977 blocks) and independently generated children clustered tightly in the 977–980 band. A single subsequent slice with the *same* flagship model but with extended reasoning

Table 1. Reasoning-mode experiment: 10-iteration slices from the frozen plateau state (best prior: 0.5423 / 977 blocks).

Slice	Reasoning	Seed	Valid iters	First improv. (slice iter)	Best score / cost	Breakthrough
ON-r1	enabled	43	10/10	8	0.8581 / **620**	yes
ON-r2	enabled	44	10/10	4	0.8581 / **620**	yes
ON-r3	enabled	45	10/10	3	0.8581 / **620**	yes
OFF-r1	disabled	43	10/10	2 (jitter +0.0003)	0.5426 / 978	no
OFF-r2	disabled	44	10/10	—	0.5423 / 977	no
OFF-r3	disabled	45	10/10	—	0.5423 / 977	no

enabled broke the plateau in its first improving iteration, producing a general affine-geometry construction that reaches exactly 620 blocks — the best-known value for $C(32, 8, 4)$ since 1996. One observation, however, cannot distinguish a causal effect of reasoning mode from a lucky draw. We therefore ran a controlled repetition experiment.

Design. Both arms resume from the same frozen evolution state (checkpoint after iteration 75; best program 0.5423/977, population of 39 programs across 4 islands). Each arm runs $k = 3$ independent slices of 10 iterations each. The arms are paired by the evolution RNG seed (43, 44, 45 for repeats 1–3 in both arms) and executed in alternating order (ON-r1, OFF-r1, ON-r2, …) so that machine-load drift affects both arms symmetrically. Everything else is held fixed: the same model (Claude Opus 5 via a local OpenAI-compatible proxy), the same system message, full-rewrite mutation mode in both arms, temperature 0.8, and the same evaluation pipeline (fitness = mean combined score over two solver seeds, 300 s solver budget per run, hard kill at 320 s).

Treatment definition. The treatment is "extended reasoning with a token budget that accommodates it". The reasoning-enabled arm runs with max_tokens = 64,000 and a 900 s request timeout, because measured reasoning traces consume ≈42k tokens before content emission; the reasoning-disabled arm retains the campaign configuration (max_tokens = 20,000, 600 s timeout). We disclose this as a bundled confound rather than attempt to equalize budgets: a reasoning call *requires* the larger budget (at 20k tokens the reasoning path produces empty content, Section 5), while the disabled arm never approached its 20k ceiling (zero truncation events across all disabled-arm calls; typical completions of 7–9k tokens). The effective budget available for *content* is therefore comparable across arms; the additional tokens in the enabled arm are consumed by reasoning itself.

6.2 Results

Table 1 summarizes the six slices. We define a *breakthrough* as any child whose verified cost falls below 900 blocks (the plateau band never left 977–1011; the observed breakthroughs all reach 620, so the threshold is not sensitive).

Every reasoning-enabled slice broke the plateau, and each reached *exactly* 620 blocks — the affine construction re-emerged independently in all three repeats, from different parents and different LLM samples. No reasoning-disabled slice produced anything outside the plateau band; the only nominal "new best" in that arm was a +0.0003 score jitter at unchanged solution quality. The disabled-arm children over all three slices (plus the 25 disabled full-rewrite iterations of Section 5, which used the same configuration from an earlier checkpoint) span 55 full-rewrite attempts without a single structural jump.

One procedural incident is disclosed for completeness: the first execution of OFF-r3 was invalidated by provider-side rate limiting (48 HTTP-429 responses; only 2 of 10 iterations completed,

neither improving). The slice was re-run under the identical configuration once quota recovered; the re-run (10/10 valid iterations, no breakthrough) is the observation reported in Table 1.

6.3 Statistical analysis

Under the null hypothesis that reasoning mode does not affect breakthrough probability, the observed split (3/3 vs. 0/3) has one-sided Fisher exact probability $p = 1/\binom{6}{3} = 0.05$. Two supporting observations lie outside the pre-registered design: the original discovery slice (reasoning-enabled, breakthrough to 620; including it gives 4/4 vs. 0/3, $p = 1/35 \approx 0.029$) and the 25 reasoning-disabled full-rewrite iterations of Section 5 (0 breakthroughs). The effect size is not marginal: every success reduced the best-known-to-us cost by 357 blocks (36%), from the plateau at 977 to the 30-year-old best-known value.

We emphasize the scope of the claim: this experiment establishes that extended reasoning was decisive *for this model, on this plateau, in this loop* — a structural-paradigm-shift setting. It does not claim that reasoning helps uniformly; published ablations in other task families report mixed and even negative effects (Section 2), which makes the categorical separation observed here noteworthy.

6.4 Cost analysis and a practical recipe

Reasoning calls are expensive: measured completions consumed ≈47.7k output tokens per call (of which ≈42k reasoning) versus 7–9k for disabled-arm calls — roughly a five-fold output-token cost, with ∼9–10 minute latency per call. The experiment thus supports a two-tier operating recipe for LLM-evolutionary search, echoing but sharpening the fast/strong model split of AlphaEvolve: run routine incremental evolution with cheap non-reasoning calls, and spend reasoning-enabled calls *only when the population plateaus and the problem plausibly demands a change of construction paradigm*. In our campaign this recipe would have saved the entire 25-iteration non-reasoning full-rewrite slice, whose 55 attempts explored the plateau's basin without ever leaving it.

7 Pre-registered Benchmark and Generalization

7.1 Protocol

To turn the single-cell result of Section 6 into a generalization claim we froze a benchmark protocol *before* executing any benchmark run (the protocol text was committed to the project repository prior to execution; the results section was appended afterwards without modifying the protocol — the commit timestamps are public).

Cells and roles. The benchmark comprises 29 cells $C(v, k, t)$ of the covering repository, partitioned into five roles: (i) two GATE cells whose optima are proven by our own exhaustive enumeration (instrument health); (ii) the single TRAINING cell $C(32, 8, 4)$, the only cell ever present in the evolutionary fitness signal — it is retained in the table for transparency but excluded from all generalization statements; (iii) fifteen AFFINE-HOLDOUT cells, prime-power universes where the discovered affine construction is applicable; (iv) two AFFINE-BOUNDARY cells where the construction applies but the archive value is strictly better than the pure construction; and (v) nine NON-PRIME-POWER cells where the affine path is inapplicable and any quality must come from the evolved generic search machinery. None of the 28 non-training cells was ever evaluated by the evolution loop.

Runs. Two pinned artifacts are compared: the pre-evolution seed solver (baseline; one run, seed 0) and the final evolved program (three runs, seeds 0/1/2 — the variance across seeds is part of the reported result). Per-cell wall-clock budgets (60–300 s) were fixed in the protocol. Solutions are scored by exact recount of covered t-subsets; every claimed tie is re-verified by a standalone verifier that uses only the Python standard library and shares no code with the harness. Ambient

Table 2. Pre-registered 29-cell benchmark. Archive = best-known value (frozen La Jolla repository, cross-checked against its live successor). Evolved column shows the range over three seeds (single number = all seeds identical). *proven optimal by our enumerator; †prime-power universe, but the affine construction is inapplicable at $(k, t) = (16, 4)$.

Cell	Role	Archive	Seed solver	Evolved	Outcome
$C(7, 3, 2)$	GATE	7*	7	7	tie
$C(13, 3, 2)$	GATE	26*	26	26	tie
$C(32, 8, 4)$	TRAINING	620	1258	620	tie (training)
$C(8, 4, 3)$	AFFINE-H	14	14	14	tie
$C(9, 3, 2)$	AFFINE-H	12	12	12	tie
$C(16, 4, 3)$	AFFINE-H	140	154	140	tie
$C(16, 8, 4)$	AFFINE-H	30	54	30	tie
$C(25, 5, 2)$	AFFINE-H	30	38	30	tie
$C(27, 3, 2)$	AFFINE-H	117	118	117	tie
$C(27, 9, 3)$	AFFINE-H	39	80	39	tie
$C(32, 4, 3)$	AFFINE-H	1240	1450	1240	tie
$C(32, 16, 5)$	AFFINE-H	62	219	62	tie
$C(32, 17, 5)$	AFFINE-H	62	156	62	tie
$C(49, 7, 2)$	AFFINE-H	56	91	56	tie
$C(49, 8, 2)$	AFFINE-H	49	72	49	tie
$C(64, 4, 3)$	AFFINE-H	10416	12986	10416	tie
$C(81, 3, 2)$	AFFINE-H	1080	1152	1080	tie
$C(81, 9, 3)$	AFFINE-H	1170	2504	1170	tie
$C(27, 10, 3)$	AFFINE-B	35	59	36	behind +1
$C(32, 18, 5)$	AFFINE-B	56	119	99–100	behind +43
$C(24, 6, 4)$	NON-PP	784	1171	1047–1049	behind +263
$C(28, 9, 3)$	NON-PP	56	90	58–70	behind +2
$C(21, 10, 3)$	NON-PP	18	25	18	tie
$C(20, 12, 4)$	NON-PP	20	29	20	tie
$C(23, 10, 3)$	NON-PP	24	34	24	tie
$C(25, 16, 4)$	NON-PP†	17	24	17	tie
$C(30, 12, 3)$	NON-PP	30	51	31	behind +1
$C(30, 9, 3)$	NON-PP	66	113	91	behind +25
$C(22, 15, 5)$	NON-PP	22	35	29–30	behind +7

machine conditions (26–41% background load from unrelated desktop processes) were recorded at launch; solver processes ran sequentially, one at a time.

7.2 Results

All 116 scheduled runs completed (297 minutes wall clock). Table 2 gives the full benchmark; no row is omitted.

Summary. The single evolved program matches the best-known value in 22 of 29 cells; all 22 ties were re-verified by the independent verifier (22/22 valid). The pre-evolution baseline is strictly worse in 27 of 29 cells (equal only on the two smallest instances). Seed stability is high: in 25 of 29 cells all three seeds returned *identical* costs, an empirical corroboration of the work-counter determinism contract imposed on evolved programs (Section 3); where variance appears it is narrow (99–100, 1047–1049, 29–30), with one exception ($C(28, 9, 3)$: 58–70).

7.3 Generalization analysis

Four independent lines support the generalization claim.

Holdout discipline. 28 of 29 cells never appeared in any fitness signal; the loop optimized against exactly one cell. The 21 non-training ties are therefore out-of-training results in the strictest sense available to an evolutionary pipeline.

Zero per-cell adaptation. All runs execute one pinned program with the same CLI, no cell-specific parameters, and pre-declared seeds.

Beyond the discovered construction. Ties are not merely replays of the affine construction. In $C(49, 8, 2)$ the pure construction yields 56 blocks while the archive value is 49; the evolved local search closes the entire gap. Four ties occur in NON-PP cells where the affine path is inapplicable — including $C(23, 10, 3) = 24$, a value that *none* of our earlier campaign artifacts had reached (best previous: 25). These ties are produced by the evolved generic machinery (incremental-gain greedy, redundancy elimination, element-exchange local search with kicks), not by the geometric template.

Regression guard. Discovering the affine paradigm did not degrade the program elsewhere. Head-to-head against the strongest pre-breakthrough artifact at equal budgets: $C(28, 9, 3)$ 70 vs. 73 (the post-breakthrough program is *better*), $C(24, 6, 4)$ 1048 vs. 1042 (within the measured single-seed noise band of anytime solvers), and both gate cells at their proven optima for both programs.

7.4 Negative results

Seven cells remain behind the archive, and we report them as first-class results. The two largest gaps are structural: $C(32, 18, 5)$ (+43) exposes the weakness of the padded-flat variant of the affine construction, and $C(24, 6, 4)$ (+263) shows that on large non-prime-power cells the evolved search machinery alone does not approach 1996-era engineered designs — mirroring, in the negative, the Section 6 finding that closing such gaps requires a construction-level insight rather than more search. Two cells miss by a single block ($C(27, 10, 3)$, $C(30, 12, 3)$); by the campaign's own precedent ($C(28, 9, 3)$ required a dedicated evolution slice to reach its archive value), focused slices may close these, but we did not run them for this paper. Cells $C(64, k, 5)$ were excluded at protocol time because the evolved program's t-subset index exceeds memory at $\binom{64}{5} \approx 7.6 \cdot 10^6$; this is declared as a scope gap, not a result.

Finally, we reiterate the honest framing of the entire section: every positive outcome above is a *tie* with a best-known value, not an improvement. Together with the archive's provenance (most of these values date to the 1990s geometric constructions), the benchmark suggests that the small stale cells of the repository are likely at or near optimal — and that genuine records in this domain will require new construction families, not better local search.

8 Contamination Analysis

The obvious objection to any rediscovery claim is that the model memorized the answer: repository tables and the founding constructions paper are public and almost certainly in the training corpus. We address the objection with six audit lines and a dissociation experiment, and we calibrate the claim accordingly.

Audit lines. (K1) The winning program contains no stored blocks, sizes, or cell-specific constants — its only dense numeric literal is a list of small primes; the construction is *derived* for arbitrary prime-power universes. (K2) No archive value ever entered the loop's inputs: prompts and configurations are grep-audited per commit, and the value 620 appears in no artifact the LLM could read. (K3) In $C(49, 8, 2)$ the pure construction yields 56 blocks while the tie requires 49; the gap was closed by the evolved local search — behavior a lookup cannot produce. (K4) Four benchmark ties occur in cells where the affine construction is inapplicable, including one value ($C(23, 10, 3) = 24$) that no earlier

Table 3. Capability dissociation.

Capability	Probe	Outcome
Recall the value 620	direct question, temp. 0	unreliable (declined)
Emit the artifact directly	64k-token reasoning, no code	failed (0 blocks)
Select paradigm, implement as code	evolution loop (Sec. 6)	reliable (3/3)

artifact of the campaign had reached. (K5) Retrieval is cheap and does not require reasoning tokens; yet the identical model failed 55/55 full-rewrite attempts with reasoning disabled and succeeded 3/3 with it enabled. A memorization mechanism would appear in both arms. (K6) Every claimed artifact is verified by exact recount; contamination could at most affect the interpretation (discovery vs. rediscovery) — and we claim only rediscovery.

Dissociation probes. We probed the deployed model directly (temperature 0). Asked for the best-known value of $C(32, 8, 4)$, the reasoning-disabled model *declined to produce a number*, stating it did not reliably know it. Asked to emit an explicit covering as plain text — no code allowed — the reasoning-enabled model consumed its entire 64k-token budget in reasoning (596 s) and emitted *zero blocks*. The same model, allowed to write code inside the loop, produced a correct general construction in every enabled repeat.

Calibrated claim. The model's latent mathematical knowledge — that affine geometries yield good coverings is textbook material — is a *permitted resource*, exactly as a human expert's literature knowledge would be. What the experiments show is that this knowledge is not accessible as retrieval: it becomes a verified artifact only through the reasoning-to-code-to-execution pathway that the harness provides and the reasoning mode unlocks. We do not claim the knowledge is absent from the weights (a null probe cannot prove absence, and the probes ran in single configurations); we claim, and demonstrate, that the pipeline — not recall — is what produces the verified object.

9 Limitations and Threats to Validity

Causal claim scope. The controlled experiment has $k = 3$ per arm (the smallest design admitting $p \leq 0.05$ on a clean split), one model family (Claude Opus 5; the engine model was never run with reasoning), one plateau, one problem. We therefore claim decisiveness of reasoning mode *in this setting*, not universality; the mixed ablation literature (Section 2) suggests the effect is regime-dependent, and characterizing that regime is future work.

Bundled treatment. Reasoning mode could not be isolated from the token budget that accommodates it (64k vs. 20k); we argue in Section 6 that the budget is a constitutive part of the treatment (without it, reasoning calls return empty content) and note that the disabled arm never approached its own ceiling.

Ties, not records. Every positive result matches a best-known value; none improves one. The benchmark suggests small stale cells sit at or near optimality, so tie-density there is informative about the *method*, not about the frontier. Where real gaps remain (Section 7), our artifacts are substantially behind, and we present this symmetrically.

Anytime variance and environment. Evolved solvers are anytime processes sensitive to machine load. We mitigated with the work-counter determinism contract and multi-seed fitness, and the benchmark shows high seed stability (25/29 identical triples), but runs executed under recorded ambient desktop load (26–41%), not a dedicated quiet machine; single-seed comparisons inherit a measured noise band of a few blocks.

Verification asymmetry of the claim itself. All quantitative claims rest on our own instrument. The instrument is calibrated against theory (Fort–Hedlund exact cells, bound invariants), against

our independent enumerator's proofs, and headline solutions are re-verified by a standalone stdlib verifier that shares no code with the harness — but all of these were written within this project; we release everything for third-party re-verification.

Contamination residuals. Section 8 dissociates retrieval from the reasoning-to-code pathway but cannot prove the absence of memorized material in model weights; probes ran in single configurations.

Engineering external validity. The ensemble-seeding pathology, waste dynamics, and proxy-level fixes are reported for one framework version (OpenEvolve 0.3.2) and one serving stack; they are offered as cautionary patterns, not as durable properties of those systems. Candidate processes were not OS-sandboxed on the experiment host.

Objective vs. application. We optimize the canonical benchmark objective (minimum block count). Real covering applications carry side constraints (balance, resolvability, implementation cost) that our objective does not measure; claims are scoped to the benchmark.

10 Conclusion

We set out to answer a question that the LLM-evolutionary discovery literature has left implicit: *which model capability does the breakthrough work?* On a covering-design campaign instrumented with mathematically exact verification at every step, the answer was sharp. Incremental evolution — cheap diff mutations, ensemble engines, even full rewrites by the same flagship model — carried the population to a genuine plateau and no further. Extended reasoning, and nothing else we varied, converted the model's latent textbook mathematics into working general code: three independent reasoning-enabled slices each rediscovered the classical affine-geometry construction and matched a value untouched since 1996, while fifty-five reasoning-disabled attempts never left the plateau's basin.

The result reframes how we read systems in the FunSearch/AlphaEvolve lineage: the evolutionary loop is the *delivery mechanism*, exact verification is the *warrant*, but on construction-shaped problems the paradigm shift itself appears to be a reasoning phenomenon — one that is neither retrievable by direct recall nor emittable without code, as our dissociation probes show. Practically, this yields a two-tier operating recipe (evolve cheaply; spend reasoning tokens only at plateaus) and a set of instrument-hygiene mechanisms (multi-seed fitness, work-counter determinism, solution archiving, leak-audited prompts) that we found necessary to make any of the above measurable.

Under a pre-registered 29-cell benchmark the single evolved program matched 22 best-known values — including four outside its discovered construction family — with every tie independently re-verified. The frontier itself did not move: no archive value was improved, and the stagnant small cells are probably near-optimal. Records in this domain will require new construction families, which suggests the next experiment in kind: pointing reasoning-enabled slices at cells whose best-known values do *not* descend from clean algebraic constructions. Our harness, certificates, protocols, raw logs, and negative results are released to make both replication and that next step possible.

Reproducibility and Artifacts

All code, certificates, frozen protocols, and raw logs are released at https://github.com/keremt-dev/discovery-harness and archived at DOI 10.5281/zenodo.21920942: the problem-agnostic harness and covering plugin (MIT license), solution certificates with a standalone stdlib verifier (CC-BY 4.0), the pre-registered benchmark protocol with its results table, the controlled-experiment slice logs, and the contamination probe transcripts. Every tie claimed in this paper can be re-verified in minutes with `python verify_cover.py v k t solution.txt`, which shares no code with the harness. The evolution runs used OpenEvolve 0.3.2 [ope 2025] with configuration files included in

the repository; model access went through a local OpenAI-compatible proxy, and we document the serving-stack behaviors (reasoning-mode payload overrides, ensemble-seeding pathology) that affect reproduction.

Acknowledgments

The problem-selection methodology of the broader discovery programme behind this paper was motivated by the published work of G. Yazgı Tütüncü and Cihangir Özkut on reliability design optimization; we thank them for that inspiration. We also thank Dan Gordon for three decades of curation of the covering repository, without which the stagnant-frontier experimental setting of this paper would not exist.

References

2025. OpenEvolve: open-source implementation of AlphaEvolve-style program evolution (v0.3.2). https://github.com/codelion/openevolve. TODO-verify: tercih edilen atif bicimi.

Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. 2025. CodeEvolve: an open-source evolutionary framework for algorithmic discovery and optimization. arXiv:2510.14150

Jay Bhan, Nicole Nobili, and Patrick Langer. 2026. New Bounds for Zarankiewicz Numbers via Reinforced LLM Evolutionary Search. arXiv:2605.01120

Davis Brown, Jesse He, Helen Jenne, Henry Kvinge, and Max Vargas. 2025. Even with AI, Bijection Discovery is Still Hard: The Opportunities and Challenges of OpenEvolve for Novel Bijection Construction. arXiv:2511.20987

Chaoying Dai, Pak Ching Li, and Michel Toulouse. 2006. A Cooperative Multilevel Tabu Search Algorithm for the Covering Design Problem. In *Artificial Evolution (EA 2005), LNCS 3871*. doi:10.1007/11740698_11

Houde Dong, Yifei She, Kai Ye, Liangcai Su, Chenxiong Qian, and Jie Hao. 2026. GMP: A Benchmark for Content Moderation under Co-occurring Violations and Dynamic Rules. arXiv:2603.01724

Marion K. Fort and Gustav A. Hedlund. 1958. Minimal coverings of pairs by triples. *Pacific J. Math.* 8 (1958), 709–719.

Daniel M. Gordon. 2026. The La Jolla Covering Repository (frozen 2026-03-01) and coveringrepository.com (live successor). doi:10.5281/zenodo.10779736

Daniel M. Gordon, Greg Kuperberg, and Oren Patashnik. 1995. New constructions for covering designs. *Journal of Combinatorial Designs* 3, 4 (1995), 269–284. arXiv:math/9502238. doi:10.1002/jcd.3180030404

Jacqueline He, Howard Yen, Margaret Li, et al. 2025. Precise Information Control in Long-Form Text Generation. arXiv:2506.06589

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution. arXiv:2509.19349

Hieu Xuan Le, Benjamin Goh, and Quy Anh Tang. 2026. Prompt Attack Detection with LLM-as-a-Judge and Mixture-of-Models. arXiv:2603.25176

Ansh Nagda, Prabhakar Raghavan, and Abhradeep Thakurta. 2026. Reinforced Generation of Combinatorial Structures: Ramsey Numbers. arXiv:2603.09172

Alexander Novikov et al. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131

Kari J. Nurmela and Patric R. J. Östergård. 1993. Upper bounds for covering designs by simulated annealing. *Congressus Numerantium* 96 (1993), 93–111. TODO-verify: cilt/sayfa.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. 2024. Mathematical discoveries from program search with large language models. *Nature* 625 (2024), 468–475. doi:10.1038/s41586-023-06924-6

Johanan Schönheim. 1964. On coverings. *Pacific J. Math.* 14 (1964), 1405–1411.

Jaward Sesay, Yue Yu, and Börje F. Karlsson. 2026. HandwritingAgent: Language-Driven Handwriting Synthesis in Scalable Vector Space. arXiv:2606.18788

Rui Zhang, Fei Liu, Xi Lin, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Understanding the Importance of Evolutionary Search in Automated Heuristic Design with Large Language Models. In *Parallel Problem Solving from Nature – PPSN XVIII*. arXiv:2407.10873. doi:10.1007/978-3-031-70068-2_12